Chapter 1: Foundations of Distributed Consensus

Distributed systems rely on state machine replication to tolerate node crashes without losing committed data. Every replica executes deterministic commands in the exact same log order; consequently, all non-faulty nodes converge on identical state.

Safety guarantees that two replicas never commit conflicting values at the same log index, even under arbitrary message delays. Liveness ensures that the cluster eventually makes progress and commits client proposals once transient network partitions heal.

The FLP impossibility theorem proves that no deterministic consensus protocol can guarantee both safety and liveness in a purely asynchronous network if even one node may crash.

Chapter 2: Leader Election and Quorum Leases

Raft decomposes distributed consensus into three cleanly separated subproblems: leader election, log replication, and safety verification. Randomized election timeouts between 150ms and 300ms prevent split votes from stalling candidate nodes indefinitely.

A candidate must collect votes from a strict majority quorum of cluster members before transitioning to leader and appending new log entries. Periodic heartbeats from the elected leader reset follower timers and suppress unnecessary disruptions across healthy replicas.

Chapter 3: Log Compaction via Snapshots

Unbounded replication logs eventually exhaust local disk capacity and dramatically slow down crash recovery replay times. Periodic snapshotting serializes the current state machine image to durable storage, allowing replicas to discard committed log prefixes safely.

When a severely lagging follower requests log entries that have already been compacted away, the leader streams an InstallSnapshot RPC containing the latest checkpoint metadata and state.